

SYSTEM ARCHITECTURE DOCUMENT (SAD)

System Name: Academic Journal SaaS Platform (Dergi Platformu Clone)

Primary Architecture Pattern: Modular Monolith with Multi-Tenant Cloud Architecture

1. PROJECT OVERVIEW

This platform is a highly specialized academic publishing system designed as a full enterprise-grade, multi-tenant Software as a Service (SaaS) infrastructure for managing global academic journals.

The backend system will be responsible for:

- Multi-tenant logical isolation (hosting multiple journals)
- Custom domain and subdomain routing
- Manuscript submission, multi-lingual metadata validation, and version control
- Double-blind peer review orchestration and conflict-of-interest filtering
- **Copyediting, typesetting, and production layout management**
- **Early Access (Article-in-Press) instantaneous publishing**
- **Manuscript withdrawal automated workflows**
- Automated plagiarism detection webhooks (iThenticate/Turnitin)
- Automated DOI generation and registration pipelines (Crossref)
- Enterprise mass mailing and academic promotion engines
- CMS and journal branding management
- SaaS subscription billing systems
- Secure, time-limited media storage for unpublished research
- High-volume event and submission state processing

2. HIGH-LEVEL SYSTEM ARCHITECTURE

- Next.js SSR Frontend (Public & Dashboards)
- NestJS API Gateway
- Auth & Tenant Service
- Journal CMS Service
- Manuscript Submission Service
- Peer Review Service
- **Production & Layout Service**
- Integrations Service (DOI, Plagiarism)
- Billing Service
- Notifications Service
- Redis + BullMQ
- Background Processing Workers
- PostgreSQL Cluster
- AWS S3 / Cloudflare R2 Storage

3. BACKEND TECHNOLOGY STACK

Layer	Technology
Framework	NestJS
Language	TypeScript
Database	PostgreSQL
Database Hosting	AWS RDS / DigitalOcean Managed
ORM / Query Builder	Prisma or TypeORM
Queue System	BullMQ
Message Broker	Redis
Authentication	JWT + ORCID OAuth
External APIs	Crossref, iThenticate, SendGrid
Storage	AWS S3 / Cloudflare R2
Logging	Pino
Monitoring	Prometheus + Grafana
Error Tracking	Sentry
Documentation	Swagger OpenAPI
Deployment	Docker + NGINX + CI/CD

4. REQUIRED BACKEND PROJECT STRUCTURE

The backend must follow a strict modular clean architecture structure.

```
src/
├─ main.ts
├─ app.module.ts
├─
├─ config/
│   ├── env.validation.ts
│   ├── configuration.ts
│   └─ constants.ts
├─
├─ common/
│   ├── decorators/
│   ├── filters/
│   ├── guards/
│   ├── interceptors/ (Crucial: BlindReviewInterceptor)
│   ├── exceptions/
│   └─ utils/
├─
├─ modules/
│   ├── health/
│   ├── database/
│   ├── redis/
│   ├── auth/
│   ├── users/
│   ├── tenants/
│   ├── journals/
│   ├── pages-cms/
│   ├── submissions/
│   ├── reviews/
│   ├── layout/
│   ├── issues/
│   ├── integrations/
│   │   ├── crossref-doi/
│   │   ├── ithenticate/
│   │   └─ orcid/
│   ├── billing/
│   ├── notifications/
│   ├── storage/
│   └─ admin/
├─
├─ workers/
│   ├── doi.processor.ts
│   ├── plagiarism.processor.ts
│   └─ email-campaign.processor.ts
```

```
|   └─ archive.processor.ts
|
|   └─ infrastructure/
|       ├── sentry/
|       ├── logger/
|       └─ storage/
|
|   └─ prisma/
|   └─ docs/
|
|   └─ tests/
|       ├── unit/
|       ├── integration/
|       └─ e2e/
```

5. CORE BACKEND RESPONSIBILITIES

The backend system must implement the following core domains handling high volume transaction scaling efficiently.

6. AUTHENTICATION & AUTHORIZATION SYSTEM

Responsibilities

- User registration and verification flows
- Login/logout management
- JWT authentication via HttpOnly cookies
- Refresh tokens management
- ORCID SSO (Single Sign-On) Integration
- Strict Role-Based Access Control (RBAC) per Tenant context

Roles

- **super_admin** (Platform Owner)
- **editor** (Executive Journal Editor)
- **layout_editor** (Typesetter & Production Manager)
- **reviewer** (Academic Peer Reviewer)
- **author** (Submitting Researcher)

Required APIs

- POST **/api/v1/auth/register**
- POST **/api/v1/auth/login**
- POST **/api/v1/auth/orcid/callback**
- POST **/api/v1/auth/refresh**
- POST **/api/v1/auth/logout**
- GET **/api/v1/auth/me**

7. TENANT & JOURNAL MANAGEMENT SYSTEM

The backend must manage multi-tenant environments dynamically, isolating individual custom scopes.

Journal Data

- id, tenant_id
- subdomain (e.g., [journalname.dergiplatformu.com](#))
- name_tr / name_en, issn / e_issn
- logo_url, cover_url, status
- settings (review_deadlines, plagiarism_threshold)

Required Features

- Tenant provisioning pipelines
- Subdomain routing and hostname resolution layers
- Mini-CMS for custom static page creation (About, Guidelines)
- Issue management & Early Access (Article-in-Press) standalone tracking configuration

APIs

- POST [/api/v1/journals](#)
- GET [/api/v1/journals](#)
- GET [/api/v1/journals/:id/settings](#)
- PUT [/api/v1/journals/:id](#)
- POST [/api/v1/journals/:id/pages](#)

8. MANUSCRIPT SUBMISSION SYSTEM

The core engine for handling academic papers and processing workflow files safely.

Required Features

- Multi-step submission wizard processing engines
- Blinded PDF upload handling and parsing
- **Dual-Language Metadata Validation:** Enforces mandatory multilingual entries (e.g., Turkish and English) for Title, Abstract, and Keywords at the DTO layer before bypassing the pre-check validation phase.
- Copyright transfer agreement signatures
- **State Machine Lifecycle tracking:** `PENDING_PRE_CHECK` → `IN_REVIEW` → `REVISION_REQUIRED` → `ACCEPTED` → `IN_COPYEDITING` → `READY_FOR_PRODUCTION` → `PUBLISHED` (or marked as `WITHDRAWN`)

APIs

- POST `/api/v1/submissions`
- GET `/api/v1/submissions`
- GET `/api/v1/submissions/:id`
- PUT `/api/v1/submissions/:id/status`
- POST `/api/v1/submissions/:id/revisions`
- POST `/api/v1/submissions/:id/withdraw` : Handles the manuscript withdrawal workflow. It completely truncates reviewer assignments, terminates pending evaluation tasks, and securely wipes unreleased drafts from private S3 buckets.

9. PEER REVIEW (DOUBLE-BLIND) SYSTEM

This is the highest priority backend domain for executing strict academic integrity workflows.

Responsibilities

- Reviewer assignment matrix and strict deadline handling
- Conflict of Interest filtering (blocking reviewers sharing the same university_id as authors)
- Evaluation form dynamic generation layers
- **Response Sanitization:** Stripping author metadata completely via NestJS interceptors if the requesting user's active role context is a reviewer.

APIs

- GET `/api/v1/reviews/pending`
- POST `/api/v1/reviews/:submissionId/assign`
- POST `/api/v1/reviews/:id/evaluate`
- GET `/api/v1/reviews/:id/blinded-manuscript`

10. PRODUCTION & EARLY ACCESS SYSTEM

Manages the final stages of article preparation and accelerated indexing models.

Responsibilities

- Assignment of accepted papers to layout_editor roles
- Version control tracking for final typeset PDFs (Galley proofs)
- **Early Access / Article-in-Press Publishing:** Decouples DOI activation from structured journal numbers, allowing accepted and proofed articles to be compiled, indexed, and available online immediately before full issue release.

APIs

- POST `/api/v1/production/:submissionId/assign-layout`
- POST `/api/v1/production/:submissionId/upload-proof`
- POST `/api/v1/production/:submissionId/publish-early`

11. INTEGRATIONS & API GATEWAY SYSTEM

The backend must orchestrate external services securely and asynchronously.

Required Features

- **Crossref API:** Automated DOI generation and metadata schema registration loops.
- **iThenticate/Turnitin:** Webhook payload listener for asynchronous plagiarism similarity score mappings.
- **SendGrid / AWS SES:** Centralized high-reputation SMTP broadcast engine for mass academic promotional distributions.

APIs

- POST [/api/v1/integrations/doi/generate](#)
- POST [/api/v1/integrations/plagiarism/webhook](#)
- POST [/api/v1/integrations/email-campaigns](#)

12. STORAGE ARCHITECTURE

Storage Layer	Purpose
PostgreSQL	Relational Metadata (Multi-Tenant Isolated Frameworks)
Redis	Caching Layers (Homepage data, Search indexes, Global system stats)
AWS S3 (Private)	Unpublished Manuscripts, Revisions, and Blinded review assets
AWS S3 (Public)	Published Open-Access finalized PDFs, Early Access proofs, Logos, and Covers

Media Security Protocols

- AWS S3 Private Bucket isolation models
- Short-lived, time-limited Presigned URLs
- Backend RBAC validation execution prior to structural URL generation

13. QUEUE & WORKER INFRASTRUCTURE

All heavy computational batch operations must run asynchronously via queues.

Worker Types

- DOI Registration Worker
- Plagiarism Check Worker
- Mass Email/Notification Campaign Worker
- PDF Copyediting / Watermarking Worker

Queue Stack: Redis + BullMQ

Attempt	Delay
1	1 minute
2	5 minutes
3	15 minutes

14. SECURITY REQUIREMENTS

- JWT Authentication mapped via secure HttpOnly Cookies
- RBAC Authorization operating contextual multi-tenant filters
- Strict DTO structural Validation execution via class-validator layers
- SQL Injection protection natively handled via abstract ORM querying layers
- XSS Protection models executed via Helmet layer configurations
- Strict Rate Limiting mechanisms enforced on high-cost Search endpoints
- Forced HTTPS connections (TLS 1.3 protocol standards)
- Bcrypt cryptography implementation for password hashing models
- **Data Sanitization Interceptors** (Enforcing critical Double-Blind metadata erasure)

15. MONITORING & OBSERVABILITY

- **Logging:** Pino Logger framework integration.
- **Error Tracking:** Sentry integration for automatic alert propagation.
- **Monitoring Stack:** Prometheus collector metrics and custom Grafana dashboards.

16. DEVOPS & DEPLOYMENT

Infrastructure Architecture

- Docker containerization models mapping localized environment configurations
- NGINX Reverse Proxy management layers configured for wildcard subdomain routing resolutions
- Let's Encrypt automated Wildcard SSL generation loops
- Horizontal Scaling strategies for decoupled state worker services

CI/CD Pipeline Sequence (Push to main branch)

Run Lint → Run Unit & Integration Tests → Build Project → Docker Image Compile → Zero-Downtime Deployment

17. REQUIRED API VERSIONING

All endpoints must explicitly follow structural prefix conventions: `/api/v1/`

18. SWAGGER DOCUMENTATION

Interactive automated Swagger documentation must expose configurations explicitly at endpoint: `/api/docs`

19. TESTING REQUIREMENTS

Test Type	Tool Selection
Unit Tests	Jest
Integration Tests	Supertest
End-to-End (E2E) Tests	NestJS Testing Module
Load / Stress Testing	k6

20. FUTURE SCALING ROADMAP

- **Phase 2:** Deconstructing monolithic domain modules into sovereign Microservice architectures + Automating XML/JATS formats compilation for global PubMed indexers.
- **Phase 3:** Integrating edge-network CDN caching layer models for Open Access PDF delivery + Executing automated AI-assisted preview summarizations.

21. FINAL DELIVERY CHECKLIST

- Auth & ORCID SSO System
- Tenant & Subdomain Routing
- Submission Workflow (with Multi-lingual validation)
- Double-Blind Review Engine & Sanitization Interceptor
- Copyediting & Production Layout System Module
- DOI Integration & Early Access Management
- Plagiarism Webhook Pipelines
- Notifications Engine

22. FINAL ENGINEERING NOTES

This backend infrastructure blueprint must be developed, tested, and maintained according to high availability guidelines, operating strictly as a unified, resilient **Multi-Tenant SaaS Infrastructure** combined with an **Enterprise Academic Publishing Engine**.